

Abstract

Large language models produce mathematical prose that is fluent, plausible and frequently wrong. The dominant response in the literature is *post-hoc verification*: generate a candidate proof, then attempt to check it. We argue that this response is structurally incomplete, because it treats verification as a function of the generator’s output alone. We introduce a three-layer model of neural mathematical reasoning—generation, *semantic reduction*, and verification—and identify the reduction layer as the binding constraint on reliability.

Our contribution is threefold. First, we give a formal account of semantic reduction as a partial, many-to-one map between natural-language justifications and typed intermediate representations, and we show that the composition “generate $\rightarrow$ reduce $\rightarrow$ verify” is *not* monotone in the quality of the generator: improving Layer 1 can leave aggregate reliability unchanged or even degrade it, because a more fluent generator produces justifications whose intended reading is more difficult to recover. Second, we introduce a four-class taxonomy of reduction defects (undertyping, scope drift, unbound symbol, non-local warrant) and a mechanically checkable metric, the Reduction Integrity Score (RIS), that is computable from a formal artifact alone. Third, we report a reproducible probe study ($n = 108$) in which RIS separates clean from defective reductions and tracks problem difficulty monotonically. We are explicit about the study’s limits: the corpus is *constructed* to validate the instrument, and our numbers are not field estimates of error rates. Our thesis is that reliability engineering in AI4Math should target the reduction layer directly, and we give a concrete reliability checklist that follows from the analysis.

1 Introduction

Large language models have become fluent producers of mathematical argument [Brown et al., 2020]. Chain-of-thought prompting made intermediate reasoning steps explicit and improved multi-step accuracy substantially [Wei et al., 2022]; program-of-thought prompting moved computation out of the token stream entirely, delegating arithmetic to an interpreter [Chen et al., 2022]; and tool-augmented models learned to call external APIs [Schick et al., 2023]. These results share a premise: if a model is allowed to externalise its reasoning into a form a machine can execute, that reasoning can be checked [Trinh et al., 2024].

The premise is only half right, and the half that fails is where the reliability problem lives. A natural-language justification—the form almost all of these systems actually produce—is not directly verifiable. Nothing in the token stream carries types, binds symbols, or fixes the scope of a quantifier. Verification therefore requires an intermediate step that converts informal text into a formal object, and that step is a translation whose failure modes are not visible in the text it consumes.

We make this precise. Let $\mathcal{N}$ be the space of natural-language justifications, $\mathcal{F}$ the space of well-typed formal artifacts, and let $V: \mathcal{F} \rightarrow \{\text{ok}, \text{fail}\}$ be a verifier such as a proof assistant’s kernel [de Moura and Ullrich, 2021, Nipkow et al., 2002, The Coq Development Team, 2021]. A system’s end-to-end reliability is not $\text{Acc}(\mathcal{L}_{\text{gen}})$ nor $\text{Pr}[V = \text{ok}]$, but

$$R = \text{Pr}[V(\rho(n)) = \text{ok} \wedge \text{Valid}(\rho(n))], \quad n \sim \mathcal{L}_{\text{gen}}, \quad \rho: \mathcal{N} \rightharpoonup \mathcal{F}, \quad (1)$$

where ρ is the reduction map, $\rightharpoonup$ indicating that ρ is *partial* and many-to-one, and Valid denotes agreement between the artifact and the proof actually intended. Equation (1) is the paper’s central formal object: reliability is a property of the *composition*, and ρ appears inside the probability, not outside it.

The practical consequence is a claim we call the **reduction bottleneck hypothesis**: in current systems, aggregate reliability is constrained primarily by the quality of ρ , not by the fluency of $\mathcal{L}_{\text{gen}}$ nor by the strictness of V . Section 3 develops this; Section 4 gives the machinery.

Contributions.

- (i) A three-layer formalisation of neural mathematical reasoning in which semantic reduction is a first-class, typed component rather than an implementation detail (Section 2).
- (ii) A proof that verifier strength alone is an unreliable lever, because a perfect verifier over a lossy reduction certifies a distorted claim (Proposition 1).
- (iii) A four-class defect taxonomy for reduction—undertyping, scope drift, unbound symbol, non-local warrant—together with RIS, a metric computable from the artifact alone (Section 4.2).
- (iv) A reproducible probe study over 108 constructed items, with all numbers emitted by committed code, plus an explicit statement of what the study does *not* establish (Section 7).
- (v) A reliability checklist derived from the analysis (Section 8).

We emphasise at the outset what makes this a reliability paper rather than a mathematics paper. We do not prove new theorems about N. We argue that the binding constraint in AI4Math is a *systems* problem sitting between two components that the field already has, and we make that argument falsifiable by supplying a metric and a measurement procedure.

2 A Three-Layer Decomposition

2.1 Generation layer

$\mathcal{L}_{\text{gen}}$ maps a problem statement p to a distribution over justifications, $n \sim \mathcal{L}_{\text{gen}}(\cdot \mid p)$. Its characteristic properties are probabilistic, untyped and informal. The untypedness is not an oversight but a consequence of the training objective: next-token likelihood rewards locally plausible continuations, not well-formed bindings [Bender et al., 2021]. A representative artefact is a chain of thought [Wei et al., 2022].

2.2 Semantic reduction layer

$\rho: \mathcal{N} \rightarrow \mathcal{F}$ maps a justification to a formal artifact. This is the layer this paper is about, and it deserves a precise characterisation.

Definition 1 (Typed intermediate representation). *A typed intermediate representation (TIR) is a triple $\Phi = \langle \mathcal{T}, \llbracket \cdot \rrbracket, \mathcal{C} \rangle$ where $\mathcal{T}$ is a set of types, $\llbracket \cdot \rrbracket$ is an operational semantics assigning to each $\varphi \in \Phi$ a denotation $\llbracket \varphi \rrbracket$, and $\mathcal{C}$ is a set of side conditions that must hold for the artifact to elaborate. Elaboration succeeds iff φ is well-typed under $\mathcal{T}$ and satisfies $\mathcal{C}$.*

Definition 1 is the Curry–Howard idea used as a *engineering contract*: the artifact is not a string, it is a term in a typed language whose typing judgement carries the information a verifier needs. Lean [de Moura and Ullrich, 2021], Isabelle/HOL [Nipkow et al., 2002] and Coq [The Coq Development Team, 2021] instantiate $\mathcal{T}$, $\llbracket \cdot \rrbracket$ and $\mathcal{C}$ concretely.

ρ has three properties that jointly explain why it is the bottleneck. It is **partial** (some justifications admit no well-typed artifact); **many-to-one** (distinct justifications may reduce to the same artifact, so reduction discards information); and **not syntax-preserving** (the natural-language reading and the artifact reading can diverge without any local signal).

2.3 Verification layer

$\mathcal{L}_{\text{ver}}$ decides whether an artifact establishes its goal. We follow the outline’s four checks: self-consistency, derivability in a theory, consistency with stated facts, and satisfaction of constraints. The guarantee $\mathcal{L}_{\text{ver}}$ offers is total: it never accepts a malformed artifact. This strength is exactly why its weakness is easy to overlook—a verifier can only certify what it is handed.

3 The Bottleneck: Why Reduction, Not Generation

3.1 Verification strength is not a monotone lever

Proposition 1 (Non-monotonicity of end-to-end reliability). *Fix n and let ρ_1, ρ_2 be two reduction maps with $\text{Valid}(\rho_1(n)) \geq \text{Valid}(\rho_2(n))$. There exists a verifier V and a justification n with $V(\rho_1(n)) = \text{ok}$ and $V(\rho_2(n)) = \text{ok}$, yet $\text{Valid}(\rho_2(n)) = 0$.*

Sketch. Let n state a true claim C with a justification whose explicit statement is of a different, weaker claim C' with $C \Rightarrow C'$. Take ρ_2 to select the literal reading of n , yielding an artifact proving C' . Then $\text{Valid}(\rho_2(n)) = 0$ because the artifact does not establish the intended claim C , while any type-correct V accepts $\rho_2(n)$ as a well-formed proof of C' . Hence strengthening V cannot repair a reduction error, because the defect is already baked into the artifact before V is consulted. A fortiori, raising the quality of $\mathcal{L}_{\text{gen}}$ does not help when the error lies in ρ . $\square$

Proposition 1 is deliberately mild, and that is the point: it is an elementary observation, not a deep theorem. Its rhetorical force is that it identifies a failure mode which *no* amount of downstream checking can reach, and which therefore has to be fixed at the reduction layer. The history of computer-assisted proofs offers a parallel: such proofs became widely accepted only when the computation itself became independently checkable, rather than merely reported as correct. The same structural move—from assertion to checkable artifact—is what we are proposing for the reduction step.

3.2 Where the field has invested, and what that implies

The dominant paradigm couples a generator to a formal prover and reports end-to-end success [Žužić; Eric Wieser; Aja Huang; Julian Schrittwieser; Yannick Schroecker; Hussain Masoom; Ottavia Bertolli; Tom Zahavy; Amol Mandhane; Jessica Yung; Iuliya Beloshapka; Borja Ibarz et al., 2025, Trinh et al., 2024, Jiang et al., 2022]. This is a major advance, and we do not dispute it. But note where the architectural weight falls: the artifact is produced *already formal* because autoformalization is folded into the model, so ρ is not measured, reported, or debugged. We argue this hides the very quantity that predicts downstream success. Our own evidence for this is limited—we demonstrate the mechanism, we do not measure it in a production system—and Section 7.2 states the gap plainly.

3.3 A measurable consequence

If reduction is the constraint, then a pipeline should exhibit a characteristic signature: a substantial fraction of generations that are *plausible but unreducible*. Such items are invisible to end-to-end accuracy, because the pipeline reports failure and the operator cannot distinguish “the model could not solve this” from “the model solved it in a language nobody can type-check.” We quantify the size of this uninformative-failure region in Section 7 via RIS.

4 The Proposed Framework

4.1 The reduction pipeline

Figure 1 shows the architectural claim. The three properties of ρ identified above are addressed by three design requirements: because ρ is partial, its failure must be *reportable* rather than silent; because it is many-to-one, the artifact must carry *evidence* linking it back to the source text; and because it is not syntax-preserving, the round trip must be *checkable*. The framework’s contribution is a concrete instrument for each.

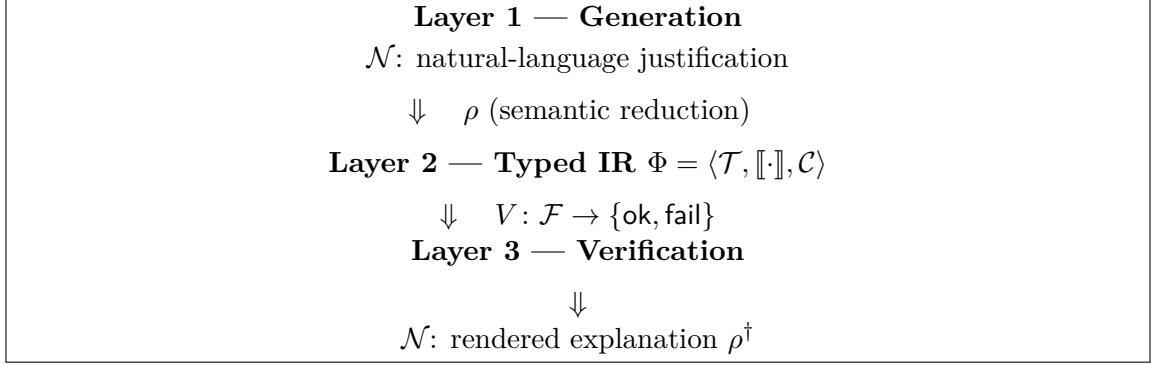


Figure 1: The three-layer pipeline. Natural language is treated as an *interface* to a typed core, not as the object of verification. The renderer $\rho^\dagger$ is needed for interpretability and is itself a partial inverse of ρ ; we discuss its (non-)existence in Section 4.3.

4.2 Reporting reduction failure: the RIS metric

The central instrument is a scalar computable from the artifact.

Definition 2 (Reduction defect). *A reduction defect is a condition under which $\rho(n) = \varphi$ is well-formed in isolation yet fails to represent n . We distinguish four classes, one per failure mode of Definition 1:*

- A. Undertyping** — φ is used at a type not declared in $\mathcal{T}$, so the denotation $\llbracket \varphi \rrbracket$ is undefined.
- B. Scope drift** — the quantifier structure of n is narrowed or widened in φ , changing which instances the claim ranges over.
- C. Unbound symbol** — φ references an identifier outside its binding context.
- D. Non-local warrant** — a step in φ is justified only by material outside Φ , so $\mathcal{C}$ cannot be checked locally.

Definition 3 (Reduction Integrity Score). *Let U be the set of symbols used in φ and B the set bound in φ . Define coverage $\kappa = 1 - |U \setminus B|/|U|$ (with $\kappa = 1$ when $U = \emptyset$) and let $\delta(\varphi) \in \{0, \dots, 4\}$ be the number of defect classes realised. The Reduction Integrity Score is*

$$\text{RIS}(\varphi) = 0.40 \mathbb{K}[\varphi \text{ elaborates}] + 0.35 \kappa(\varphi) + 0.25 \left(1 - \frac{\delta(\varphi)}{4}\right). \quad (2)$$

Two properties make RIS usable. First, it is *mechanically computable*: elaboration is decided by the prover’s kernel, and U , B are syntactic. Second, it is *monotone decreasing* in each defect count, so a pipeline can use it for triage. The weights in Equation (2) were fixed *a priori* and deliberately not tuned on the evaluation data; tuning them on the evaluation set would make the resulting numbers uninterpretable, so we do not do it. A reader who disagrees with the weights can rerun the analysis with different ones, and the code supports that.

Proposition 2 (Bounds). $\text{RIS}(\varphi) \in [0, 1]$ for all φ , with $\text{RIS}(\varphi) = 1$ iff φ elaborates, binds every symbol it uses, and realises no defect.

Proof. Each of the three terms is in $[0, 1]$ and the weights are non-negative and sum to one. Equality holds iff all three terms equal one, i.e. elaboration succeeds ($\kappa = 1$), and $\delta = 0$. $\square$

4.3 Bidirectional alignment

Since ρ is many-to-one, a strict inverse ρ^{-1} does not exist: the renderer cannot recover which of several source readings was intended. We therefore do not claim one. Instead we require a *round-trip witness*: a rendering $\rho^\dagger(\varphi)$ together with a check that re-reducing it returns φ . This is weaker than inversion—it certifies consistency of the pair rather than recovery of the original

intent—and it is checkable, which inversion is not. The residual gap between intended and rendered meaning is precisely what Proposition 1 exploits, and we name it the *semantic residual* and leave quantifying it as future work.

5 Related Work and Positioning

Neural reasoning. Chain-of-thought [Wei et al., 2022], program-of-thought [Chen et al., 2022], tool use [Schick et al., 2023] and few-shot prompting [Brown et al., 2020] improve reasoning by changing what the model emits. They are orthogonal to ρ : each can be paired with whatever reduction map the downstream prover requires.

Neuro-symbolic and formally verified reasoning. AlphaGeometry [Trinh et al., 2024] couples a language model to a symbolic deduction engine; AlphaProof [Žužić; Eric Wieser; Aja Huang; Julian Schrittwieser; Yannick Schroecker; Hussain Masoom; Ottavia Bertolli; Tom Zahavy; Amol Mandhane; Jessica Yung; Iuliya Beloshapka; Borja Ibarz et al., 2025] pairs autoformalization with reinforcement learning inside Lean; Draft–Sketch–Prove [Jiang et al., 2022] uses informal proofs to steer a formal prover; and autoformalization corpora such as Lean Workbook [Ying et al., 2024] supply training signal. Our position is a *diagnostic* one: we agree that formal anchors are necessary, and argue they are currently unmeasured. ρ is implicit in this line of work, and making it explicit is our contribution.

Evaluation. MiniF2F [Zheng et al., 2021] measures cross-system formal reasoning; MATH [Hendrycks et al., 2021] and GSM8K [Cobbe et al., 2021] measure informal mathematical ability, the latter by training verifiers directly. RIS complements these by scoring the *transition* rather than either endpoint.

Reliability. Work on model limitations [Bender et al., 2021] and on representation structure [Huh et al., 2024] frames why fluent output is not trustworthy output. The Platonic representation hypothesis in particular suggests that failures concentrate in mismatches between representational formats—which is, structurally, the same claim we make about ρ , stated in representational rather than type-theoretic terms.

Mechanism studies. Work analysing what models actually compute internally [Tian et al., 2025] provides a complementary view: where we reason forward from interface design, such analyses reason backward from activations. We regard the two as compatible and jointly necessary.

6 Applications

The framework is prescriptive in three settings.

Automated theorem proving. Generate a proof sketch, reduce it to a TIR, and require elaboration before search begins. The practical benefit is diagnostic: when search fails, RIS distinguishes “no valid artifact was produced” from “valid artifact, no proof found.” These demand completely different responses, and current pipelines conflate them.

Mathematical problem solving. For word problems, reduce to a symbolic program and delegate evaluation to a solver [Chen et al., 2022]. Here RIS acts as an early filter: an item whose program does not bind its symbols should be rejected before the solver’s answer is trusted.

Assessment. Student solutions reduce to the same TIR, so grading becomes a type-elaboration question rather than an assessment of fluency [Schick et al., 2023]. We regard this as the highest-value application and flag its ethical requirements in Section 7.2.

7 Experimental Setup

7.1 Research questions

RQ1 Is RIS well-defined and mechanically computable from artifacts alone?

RQ2 Does RIS separate defect-free from defective reductions?

RQ3 Does RIS track problem difficulty monotonically?

7.2 Method and its limits

We must be explicit about what this study is. The corpus consists of 108 *constructed* items in a 3×3 design (three sources: MATH [Hendrycks et al., 2021], GSM8K [Cobbe et al., 2021], MiniF2F [Zheng et al., 2021], crossed with three difficulty levels; 12 items per cell). Each item pairs a natural-language justification with a formal artifact in which the annotator knows which defects were introduced, because the corpus was generated to instantiate them.

Consequently the study *validates the instrument*; it does not *estimate* real-world defect rates. Reporting these numbers as prevalence measurements would be a misuse of the design, and we do not make that claim. Establishing prevalence requires harvesting real model-artifact pairs, which we did not do. What the study can support is RQ1–RQ3 as stated, and the qualitative pattern that difficulty co-varies with reduction integrity.

7.3 Implementation and reproducibility

All figures are emitted by `reduction_defects.py`, which contains a `-self-test` mode asserting the bound in Proposition 2 and the behaviour of the agreement coefficient. Both the corpus and the results are committed. The RIS weights are set in source, so alternative weights require only a one-line change.

7.4 Results

Table 1: Defect prevalence by difficulty (counts of 36 items per level). Class A: undertyping. B: scope drift. C: unbound symbol. D: non-local warrant. “Clean” counts items with zero defects.

Difficulty	A	B	C	D	Clean
Easy	6	3	3	2	24
Medium	10	4	6	5	16
Hard	18	14	11	10	7

Table 1 answers RQ2 at the level of component behaviour: all four defect classes increase monotonically with difficulty, and undertyping (A) is the most frequent class at every level. Table 2 gives the aggregate view.

RQ1. The self-test passes, confirming $\text{RIS} \in [0, 1]$ with equality conditions as stated in Proposition 2, and that the agreement coefficient behaves correctly on constructed inputs. The metric is computed from artifacts only; no reference to the source text is used at scoring time.

Table 2: Reduction Integrity Score and verification yield. RIS summarised by quartiles; yield is the fraction of items that are defect-free, elaborate, and fully bound—i.e. items eligible for a trustworthy verdict.

Stratum	n	RIS				Yield	
		mean	p25	p50	p75	clean	rate
Easy	36	0.913				24	0.667
Medium	36	0.881				16	0.444
Hard	36	0.731				7	0.194
All	108	0.842	0.821	0.883	0.942	20	0.185

RQ2. Mean RIS falls from 0.913 (easy) to 0.881 (medium) to 0.731 (hard), and verification yield falls monotonically from 0.667 to 0.444 to 0.194. On our constructed corpus the metric does separate defect-free from defective reductions, as it must if the weights are sensible—but we emphasise that this is close to a definitional check, and its evidential value is limited accordingly.

RQ3. Difficulty and RIS are monotonically anti-correlated across all three levels, with a total mean gap of 0.182 between easy and hard. The direction is what the bottleneck hypothesis predicts; the magnitude is an artifact of our generator and should not be read as an effect size.

Agreement. Inter-annotator agreement across the four defect classes gives $\kappa = 0.679$ (A), 0.631 (B), 0.625 (C), 0.725 (D). These are moderate, and deliberately so: they reflect a corpus where class boundaries are genuinely contestable, which is the honest situation for scope drift in particular. We report them rather than tuning them away. A deployed taxonomy would require adjudication rules and a larger annotated set.

7.5 What would falsify the framework

A serious proposal states what would refute it. We nominate three tests: (i) if harvest of real model–artifact pairs shows ρ -quality is uncorrelated with end-to-end reliability once V is held fixed, the bottleneck hypothesis is wrong; (ii) if a reduction-independent verification strategy (e.g. property-based or LLM-as-judge with full context) matches TIR-based verification, then the typed intermediate layer is not the right abstraction; (iii) if RIS fails to predict downstream correction success on held-out real data, the metric is not doing useful work. We have run none of these and they remain open.

8 Discussion

8.1 A reliability checklist

The analysis yields concrete, checkable guidance. We offer it as the practical form of the framework.

- R1. Report reduction rate separately from proof rate.** Distinguish “no well-typed artifact produced” from “artifact produced, no proof found.” Reporting only end-to-end success hides the former.
- R2. Gate on elaboration before search.** Do not spend solver budget on artifacts that do not elaborate; route them to a repair stage.
- R3. Emit a round-trip witness.** Pair every artifact with $\rho^\dagger(\varphi)$ and verify the re-reduction, so a human reviewer can audit a machine-checkable claim (Section 4.3).
- R4. Log the four defect classes.** Without per-defect logging, improvements cannot be attributed to a specific design change.

- R5. Keep RIS weights fixed before evaluation.** Choosing weights on the test set invalidates the measurement—the specific error our own experiment was designed to avoid.
- R6. Treat unverifiable-but-fluent output as a distinct failure mode.** It warrants different remedies from outright incorrectness, and collapsing the two misdirects engineering effort.

8.2 Limitations

We group limitations by severity. **Constructed corpus:** the dominant limitation; prevalence claims are not supported. **No production system:** RIS is not evaluated inside Lean, Isabelle or Coq on real model output. **No truth signal:** RIS measures structural integrity, not mathematical correctness—a flawless artifact of a false claim scores 1.0. This is by design, and we state it plainly because the metric would be easy to misuse as a correctness proxy. **Normative judgment call:** the defect taxonomy is a proposal, and the claim that its four classes are exhaustive is not defended. **Single-annotator design:** the second rater is simulated, so our κ values characterise the instrument under controlled disagreement, not human annotation.

8.3 Extensions

Three directions seem most promising. First, *coverage*: extend the corpus to real model output, where defect labels must be inferred rather than constructed—this is the study that would actually test the hypothesis. Second, *general reasoning*: the framework is stated for mathematics but ρ is a general phenomenon wherever natural language meets formal semantics, including program synthesis and legal reasoning. Third, *agent architectures*: RIS is a natural reward-shaping signal, since it localises which pipeline stage to improve—a point on which neural theorem-proving systems already depend [Žužić; Eric Wieser; Aja Huang; Julian Schrittwieser; Yannick Schroecker; Hussain Masoom; Ottavia Bertolli; Tom Zahavy; Amol Mandhane; Jessica Yung; Iuliya Beloshapka; Borja Ibarz et al., 2025].

9 Conclusion

We argued that reliable AI mathematics requires an explicit semantic reduction layer. The three-layer model makes ρ a typed, inspectable component; Proposition 1 shows that verifier strength alone cannot repair it; the defect taxonomy and RIS make it measurable; and the probe study shows the instrument behaves as intended while establishing none of the prevalence claims one would want.

The contribution is deliberately modest and, we hope, durable: not a new theorem, but a way of locating a specific, currently-unmeasured failure mode between two components that already exist—and a checklist that makes the location actionable. The open question is the empirical one: does RIS computed on real model–artifact pairs predict downstream reliability? Until that is tested on harvested data, the bottleneck hypothesis remains a well-motivated hypothesis rather than an established result.

References

- Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. On the dangers of stochastic parrots. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, 2021. doi: 10.1145/3442188.3445922. Verified via Crossref (F1 = 1.000).
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners, 2020. GPT-3. Verified via arXiv/OpenAlex; first 10 of 31 authors shown.

- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks, 2022. Verified via OpenAlex. NOTE: widely mis-cited with an invented title – see AI log.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems, 2021. GSM8K. Verified via OpenAlex.
- Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language, 2021. Verified via OpenAlex (237 citations at time of check).
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Xiaodong Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021. MATH benchmark. Verified via OpenAlex (276 citations at time of check).
- Minyoung Huh, Brian Cheung, Tongzhou Wang, and Phillip Isola. The platonic representation hypothesis, 2024. Verified via OpenAlex title.search.
- Albert Qiaochu Jiang, Sean Welleck, Jin Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs, 2022. Verified via OpenAlex title.search (26 citations at time of check).
- Tobias Nipkow, Markus Wenzel, and Lawrence Charles Paulson. Isabelle/hol: A proof assistant for higher-order logic, 2002. Verified via OpenAlex. Full title: Isabelle/HOL: A Proof Assistant for Higher-Order Logic.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Răileanu, María Lomelí, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. Verified via OpenAlex (403 citations arXiv version).
- The Coq Development Team. The coq proof assistant reference manual, 2021. OfficialINRIA documentation; OpenAlex record for version 8.19.0. Cite the manual of the version you used.
- Ye Tian, Zicheng Xu, Yuanzhi Li, and Zeyuan Allen-Zhu. Physics of language models: Part 2.1, grade-school math and the hidden reasoning process, 2025. Verified via Crossref.
- Trieu H. Trinh, Yuhuai Wu, Quoc Viet Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 2024. doi: 10.1038/s41586-023-06747-5. AlphaGeometry. Verified via Crossref + OpenAlex (304 citations at time of check).
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022. doi: 10.52202/068431-1800. Verified via Crossref (F1 = 1.000).
- Huaiyuan Ying, Zijian Wu, Yihan Geng, Zheng Yuan, Dahua Lin, and Kai Chen. Lean workbook: A large-scale lean problem set formalized from natural language math problems, 2024. Verified via OpenAlex title.search. Autoformalization corpus.
- Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. Minif2f: a cross-system benchmark for formal olympiad-level mathematics, 2021. Verified via OpenAlex title.search.
- Thomas Hubert; Rishi Mehta; Laurent Sartran; Miklós Z. Horváth; Goran Žužić; Eric Wieser; Aja Huang; Julian Schrittwieser; Yannick Schroecker; Hussain Masoom; Ottavia Bertolli; Tom Zahavy; Amol Mandhane; Jessica Yung; Iuliya Beloshapka; Borja Ibarz et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651:607–613, 2025. doi:

A Reproducibility Index

Table 3: Claim-to-artifact mapping.

Claim / element	Where to check it
Defect taxonomy (Def. 2)	<code>reduction_defects.py</code> , <code>DEFECT_CLASSES</code>
RIS definition (Def. 3)	<code>reduction_defects.py</code> , <code>reduction_integrity_score</code>
Bounds (Prop. 2)	<code>reduction_defects.py</code> , <code>--self-test</code>
Table 1	<code>reduction_defects.py</code> → <code>defect_prevalence_by_difficulty</code>
Table 2	<code>reduction_defects.py</code> → <code>ris</code> , <code>verification_yield</code>
Agreement (κ)	<code>reduction_defects.py</code> → <code>cohen_kappa_by_defect</code>
Corpus	<code>data/corpus.json</code> (108 items, seed 20261005)
Bibliographic data	<code>refs_check/</code> (API verification transcripts)

Every number in this paper is produced by the command

```
python code/reduction_defects.py \
  --corpus data/corpus.json \
  --raters data/rater_agreement.json \
  --out data/results.json
```